

Implementation of Atomicity and Durability

Atomicity and durability are ACID properties that ensure the reliability and consistency of transactions in DBMS

Atomicity:

It is one of the key characteristics of transactions in database management systems (DBMS), which guarantees that every operation within a transaction is handled as a single, indivisible unit of work.

Either all the changes made by a transaction are committed to the database or none of them are committed.

Durability:

Durability, guarantees that changes made by a transaction once it has been committed are permanently kept in the database and will not be lost even in the event of a system failure or crash.

The recovery-management component of a database system can support atomicity and durability by a variety of schemes. We first consider a simple, but extremely inefficient, scheme called the *shadow copy scheme*.

In the shadow-copy scheme, a transaction that wants to update the database first creates a complete copy of the database. All updates are done on the new database copy, leaving the original copy, the shadow copy, untouched. If at any point the transaction has to be aborted, the system merely deletes the new copy. The old copy of the database has not been affected.

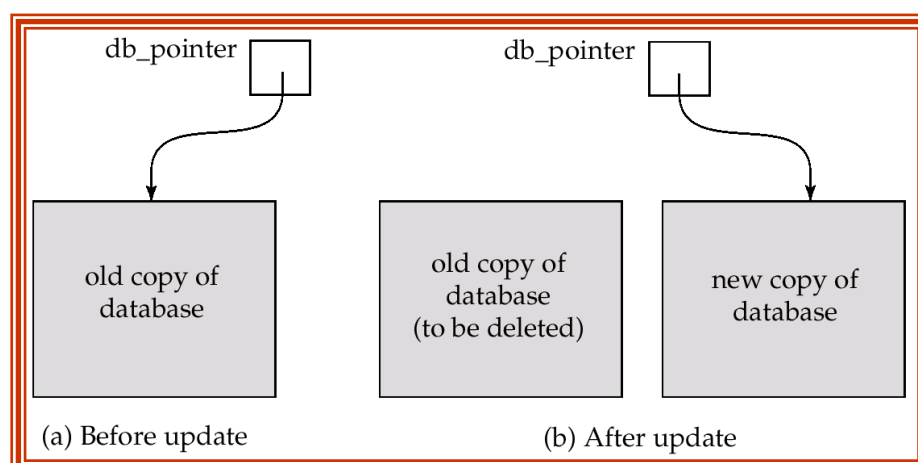
This scheme is based on making copies of the database, called shadow copies, assumes that only one transaction is active at a time. The scheme also assumes that the database is simply a file on disk. A pointer called db-pointer is maintained on disk; it points to the current copy of the database.

If the transaction completes, it is committed as follows:

First, the operating system is asked to make sure that all pages of the new copy of the database have been written out to disk. (Unix systems use the flush command for this purpose.)

After the operating system has written all the pages to disk, the database system updates the pointer db-pointer to point to the new copy of the database; the new copy then becomes the current copy of the database. The old copy of the database is then deleted.

The transaction is said to have been committed at the point where the updated db-pointer is written to disk.



We now consider how the technique handles transaction and system failures.

Maintaining Automicity

- ☐ First, consider transaction failure. If the transaction fails at any time before db-pointer is updated, the old contents of the database are not affected.
- ☐ We can abort the transaction by just deleting the new copy of the database.
- ☐ Once the transaction has been committed, all the updates that it performed are in the database pointed to by db-pointer.
- ☐ Thus, either all updates of the transaction are reflected, or none of the effects are reflected, regardless of transaction failure.

Maintaining Durability

- ☐ Suppose that the system fails at any time before the updated db-pointer is written to disk. Then, when the system restarts, it will read db- pointer and will thus see the original contents of the database, and none of the effects of the transaction will be visible on the database.
- ☐ Next, suppose that the system fails after db- pointer has been updated on disk. Before the pointer is updated, all updated pages of the new copy of the database were written to disk.
- ☐ Again, we assume that, once a file is written to disk, its contents will not be damaged even if there is a system failure. Therefore, when the system restarts, it will read db-pointer and will thus see the contents of the database after all the updates performed by the transaction.